Exploratory Data Analysis

Describing, questioning, and understanding a dataset on its own terms: what to look at, what to ask, and what you are entitled to conclude.

EDA is not chart-making; it is evidence-gathering. Every plot and statistic should answer a question or generate one. If a figure does neither, it is decoration, so delete it and keep going.

The questions EDA answers

- What does this data represent?
- How was it produced, and by whom?
- Is it trustworthy?
- What is typical, and what varies?
- What moves with what?
- What is surprising here?
- What can this data *not* tell me?

What EDA produces

Not just a notebook full of plots. The deliverables are:

- A data dictionary you actually trust
- A catalog of known defects
- A described distribution for every variable that matters
- A short list of candidate hypotheses
- An explicit list of open questions

Three modes, kept distinct

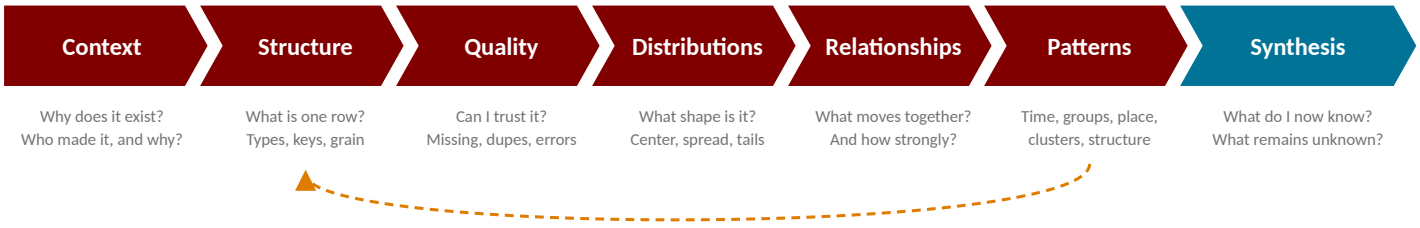
Descriptive: what is in this dataset. Claims stop at the data's edge.

Exploratory: what patterns *might* exist. Generates hypotheses; cannot confirm them.

Confirmatory: testing a pre-specified claim. Requires a hypothesis fixed *before* you looked.

A The EDA Workflow

Seven stages, though you will walk them out of order



Every finding sends you backwards. Good EDA is **iterative**, not linear.

Read it as a ratchet, not a pipeline: a quality problem found at stage 5 invalidates the distributions you read at stage 4. Re-run, don't patch.

1 Start with the question, not the data

Before a single line of code

FRAME THE INQUIRY

- Who collected this, when, and for what purpose?
- What question brought me to this data?
- What would a satisfying answer look like?
- What is the unit of analysis?
- Which comparison is the interesting one?

SET THE BOUNDARIES

- What claims could this data support?
- What claims will it never support?
- What would change your mind?
- What do I hope to find, and how do I guard against finding it anyway?

B The first ten minutes

Orientation, before any hypothesis

```
df.shape; df.dtypes; df.head(20); df.tail(20)
df.describe(include="all").T
df.isna().mean().sort_values(ascending=False)
df.nunique().sort_values()      # constants? keys?
df[date_col].agg(["min","max"])
```

WHAT THOSE FIVE LINES GIVE YOU

- **1** Row and column counts, each column's dtype, and both ends of the file
- **2** Per-column statistics: mean, SD, quartiles for numerics; top value for text
- **3** The share of values missing in each column, worst first
- **4** Distinct values per column: 1 is a constant, a row-count match is a key
- **5** The date range the file actually covers

FIVE WAYS EDA QUIETLY FAILS

- **Plot-first drift:** forty charts, no findings written down
- **Confirmation search:** looking only where you expect it
- **Ignoring provenance:** analyzing values of unknown origin
- **Summary-only reading:** a mean you never plotted
- **Silent cleaning:** dropping rows without recording why

2 Understand the structure

Grain, types, keys

```
df.shape
df.head()
df.info()
df.describe(include="all")
df.dtypes
df.nunique()
df.memory_usage(deep=True)
```

Establish, in writing:

- Rows, columns, memory footprint
- Type of each variable: numeric, categorical, ordinal, text, datetime, geospatial
- Primary keys and identifiers
- Which variables are measured, and which are derived
- Time variables and their time zone
- Hierarchy and nesting
- Repeated observations per entity
- Joins to other tables, and their cardinality

The question that gates everything

What does one row represent?

One patient? One patient-visit? One patient-visit-day? If you cannot answer in a sentence, stop and investigate. Nearly every silent analysis error (double counting, broken joins, averages of averages) begins with a misread grain.

Types lie

A ZIP code stored as `int64`, a Likert scale read as continuous, dates parsed as strings, an ID that pandas helpfully summed. Fix dtypes before you plot anything.

3 Assess data quality

Trust, then verify

MISSING DATA

```
df.isna().mean().sort_values(ascending=False)
df.isna().corr()                # do holes co-occur?
df.isna().groupby(df["segment"]).mean()
```

Ask how much is missing, whether it concentrates in particular groups or periods, and whether missingness is itself informative. Plot it: a missingness matrix or a heatmap of nulls by month often explains more than the percentages do.

Describe before you fill

Filling holes with a mean shrinks variance, distorts correlations, and invents certainty you do not have, and it does it silently. In EDA, first *characterize* the missingness; only then decide whether any filling is defensible for the question at hand.



Holes fall anywhere, unrelated to observed or unobserved values.
Dropping rows costs power, not validity.

Holes cluster in rows flagged by a variable you can see (blue).
Conditional imputation is defensible.

Holes depend on the hidden value itself (here: larger values go missing).
Imputation biases you. Model the mechanism.

Maroon = observed, gray = missing. The three mechanisms are untestable from the data alone; MAR versus MNAR is a claim about the world, so argue it from domain knowledge, not from a chart.

DUPLICATES

```
df.duplicated().sum()
df.duplicated(subset=key_cols).sum()
```

Decide which kind you have: a data error, a legitimate repeated observation, or several measurements of the same entity. Only the first is a defect; the other two are telling you that your grain is not what you assumed.

IMPOSSIBLE & PLACEHOLDER VALUES

- Negative ages, durations, or counts
- Dates before the system existed, or in the future
- Percentages above 100; probabilities above 1
- Units mixed across rows (kg and lb, USD and local)
- Sentinels masquerading as data: `999`, `-1`, `0`, `"Unknown"`, `1900-01-01`

Range checks come from domain knowledge, not from the data. Code cannot tell you that a 240 mmHg reading is a typo.

4 Describe one variable at a time

Shape before relationship

NUMERICAL VARIABLES

Report center, spread, and shape together, since a mean without a distribution is a rumor.

Center: mean, median · **Spread:** SD, IQR, range
Shape: skewness, kurtosis, modality · **Tails:** 1st/99th percentiles
Mass: share at zero, share at the min or max

```
p = [.01,.05,.25,.5,.75,.95,.99]
df["x"].describe(percentiles=p)
df["x"].plot.hist(bins="auto")  # then log scale
df["x"].plot.kde()
```

Histogram for shape, ECDF for percentiles and comparisons, boxplot only for side-by-side groups, because it hides bimodality entirely.

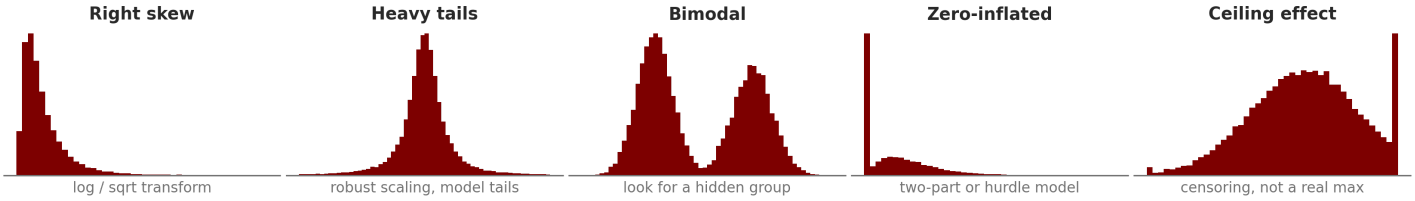
CATEGORICAL VARIABLES

```
df["category"].value_counts(dropna=False)
df["category"].value_counts(normalize=True)
```

- **Rare levels:** how many observations sit behind that 3%?
- **High cardinality:** a "category" with 4,000 levels is really an identifier or free text
- **Inconsistent labels:** `Chicago`, `CHICAGO`, `chicago` are one category wearing three hats
- **Hidden missingness:** `"N/A"`, `""`, `"none"` are nulls in costume
- **Ordinal structure:** preserve the order, and question whether the spacing is equal

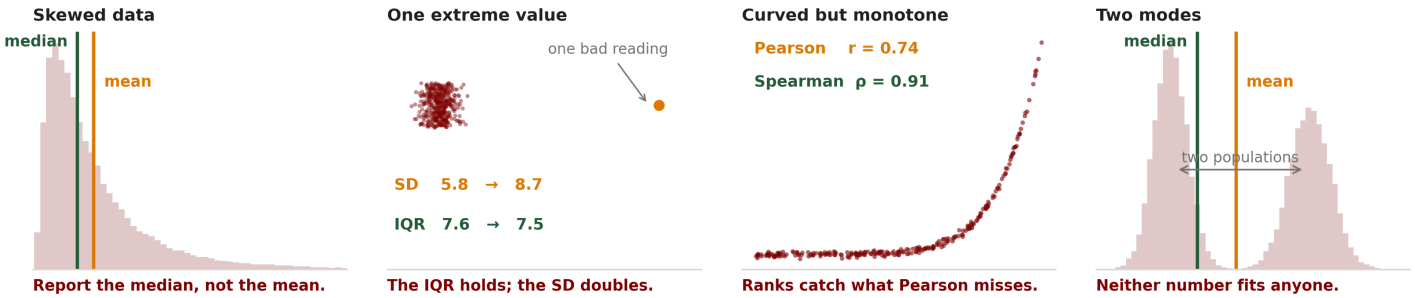
> **Always plot before you summarize**
>
> Wildly different distributions share the same mean, SD, and correlation. Summary statistics compress; plots reveal what was compressed away.

FIVE SHAPES WORTH RECOGNIZING ON SIGHT



The caption under each panel is what the shape implies, not a label. Also watch for a spike at a single value (a default or sentinel) and for discreteness masquerading as continuity.

WHICH SUMMARY STATISTIC, AND WHEN IT LIES



Rule of thumb: symmetric and unimodal → mean and SD. Skewed or outlier-prone → median and IQR, plus the percentiles that matter. Multimodal → describe the modes; no single number is honest.

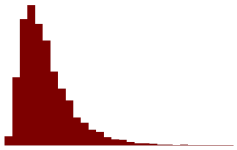
OUTLIERS

An outlier is a question, not a defect. Ask which it is: a recording error, a genuine extreme, a different population that leaked into the sample, or a heavy-tailed variable behaving normally. Fraud, equipment failure, and outbreaks live in the tail, so a rule that deletes outliers deletes the phenomenon. In EDA the job is to *find*, *examine*, *and explain* them; report your analysis with and without them if their influence is large.

5 Choose the view that answers the question

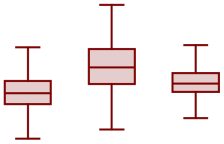
Start from the question, not the chart menu

How is it distributed?



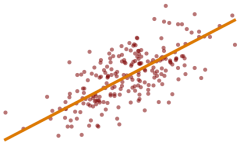
histogram · KDE · ECDF

How do groups compare?



box · violin · strip

Do two numbers relate?



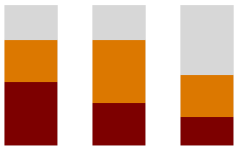
scatter + smoother · hexbin

How many per category?



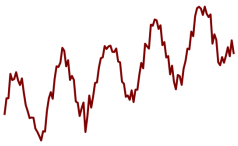
ranked bar chart

What share of the whole?



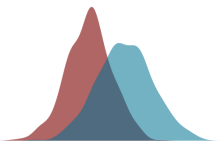
100% stacked bar · mosaic

What happens over time?



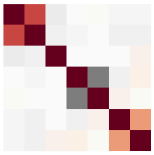
line · rolling mean · decomposition

Different shape by group?



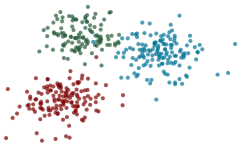
overlaid density · ridgeline

Which variables are related?



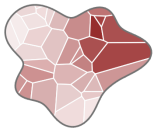
correlation heatmap · pair plot

Are there subgroups?



colored scatter · PCA / UMAP view

Where is it concentrated?



choropleth · point map · hex bin

Each panel is a question, not a chart type. If you cannot say which question a figure answers, it does not belong in your write-up.

6 The craft of an exploratory plot

Fast and honest beats pretty

MAKE IT SHOW THE DATA

- **Fix overplotting:** transparency, small points, hexbin, or 2-D density once n runs past a few thousand
- **Vary the bin width:** a histogram's story changes with the bins; try several before believing one
- **Try a log scale** whenever a variable spans orders of magnitude
- **Sort by value**, not alphabetically, when ranking categories
- **Show n** for every group you compare

KEEP IT HONEST

- Start bar charts at zero; dot plots and lines may be truncated, but say so
- Use one consistent color meaning across every figure in a project
- Do not use color to encode something a position could encode better
- Label the axes with units, including the denominator of any rate
- Pick colors that survive grayscale printing and common color-vision deficiencies

COMPARE LIKE A SCIENTIST

- **Small multiples** beat one crowded chart: same scales, repeated panel
- **Order panels meaningfully:** by size, time, or geography, never by accident
- **Overlay a reference:** the overall distribution, last year, or the null expectation
- **Aggregate last.** Look at the raw points, then the summary; averages hide the shape that produced them

Exploratory figures are for you; explanatory figures are for your reader. Make fifty rough ones quickly, then rebuild the three that mattered, with a title that states the finding, annotation on the point that carries it, and everything else stripped out.

7 Explore relationships between variables

Pick the tool from the type pair

Variable pair	Look first	Then quantify	Traps
Numeric × Numeric	Scatterplot with a LOWESS line; hexbin or 2-D density when n is large; pair plot for a few variables	Pearson *r* (linear only), Spearman / Kendall (monotone), mutual information (any dependence)	$r \approx 0$ with obvious curvature; a single point driving the correlation; overplotting hiding the mass of the data
Categorical × Numeric	Boxplots or violins side by side; overlaid ECDFs; strip plot when groups are small	Grouped mean, median, SD and **count**; difference in medians with a confidence interval; Cohen's *d* or rank-based effect size	Groups of size 3 driving a conclusion; unequal variances; a lurking third variable
Categorical × Categorical	Row-normalized crosstab; stacked or grouped bars; mosaic plot	Chi-square; Cramér's V for strength; standardized residuals to find the driving cells	Sparse cells invalidating the test; high cardinality producing noise that looks like structure
Many × Many	Clustered correlation heatmap; pair plot on a sampled subset	Correlation clusters; PCA loadings to see which variables move as a block	Reading a heatmap of 200 variables as if every cell were a finding

Correlation is not causation, and zero correlation is not independence. Pearson's *r* captures only the linear part; a perfect U-shape gives $r = 0$. Look at the scatterplot before trusting any coefficient.

Beware Simpson's paradox. A relationship that holds in every subgroup can reverse when pooled. Before believing an aggregate pattern, split it by the obvious groupings and look again.

8 Compare groups carefully

Exploration generates hypotheses; it does not confirm them

Useful tools: **t-test** and **Mann–Whitney U** (two groups), **ANOVA** and **Kruskal–Wallis** (several groups), **chi-square** (association), **Kolmogorov–Smirnov** (whole distributions), **Levene's** (equal variance), **bootstrap intervals** (almost anything).

Each carries assumptions (independence above all) that grouped, spatial, and repeated-measures data routinely violate. Check the assumption before quoting the number.

The multiplicity problem

Testing everything against everything guarantees discoveries. Twenty comparisons at $\alpha = .05$ produce roughly one "significant" result from pure noise. In EDA, treat every p-value you did not plan for as a flag for further work, not as a result.

Report the effect, the uncertainty, and the practical meaning, not just the p-value. With a million rows a 0.2% difference is significant and meaningless; with fifty rows an important effect can miss the threshold entirely.

ALWAYS ASK

- How big is the difference in units someone cares about?
- How uncertain is it, and how small are the groups?
- Does the whole distribution shift, or only its tail?

THEN ASK

- Who is in each group, and are they comparable at all?
- What third variable could produce this difference?
- Would it survive if I defined the groups slightly differently?

BEFORE BELIEVING IT

- Did I decide on this comparison before or after seeing the data?
- How many other comparisons did I look at?
- What would confirm it: new data, a new period, another site?

9 Look beyond pairs

Where real datasets get interesting

REDUNDANCY

Cluster the correlation matrix rather than reading it cell by cell. Blocks of mutually correlated variables usually mean several columns are measuring one underlying thing.

A wide table with heavy redundancy has far fewer independent dimensions than columns, which is worth knowing before you describe it as rich.

CONDITIONAL STRUCTURE

Ask whether the relationship between *X* and *Y* changes across levels of *Z*. Facet the scatterplot by *Z*, or plot the group-wise slopes together.

Diverging slopes are an interaction; a reversal is Simpson's paradox arriving in person.

PROJECTIONS

PCA shows the main axes of variation: read the scree plot for how many dimensions matter and the loadings for what they mean.

t-SNE and **UMAP** show local neighborhood structure. Scale variables first; results shift with the settings.

Treat clusters and embeddings as sketches, not evidence

t-SNE and UMAP will produce clean, well-separated blobs from pure noise, and k-means will return exactly the *k* clusters you asked for whether or not they exist. Distances between clusters and their relative sizes are not interpretable. A visible group is a *hypothesis*: check it survives different seeds, settings, and subsamples, and above all that it corresponds to something outside the plot.

10 Awkward variable types

Where the defects hide

FREE TEXT

- Plot the length distribution: empty strings and truncation show up immediately
- Count tokens; look at the most common values, and the rarest
- Normalize case, whitespace, and accents before counting anything
- Check the encoding: mojibake is a data-quality problem, not a display quirk

DATES & TIMES

- Parse explicitly; never let the parser guess day-first versus month-first
- Establish the time zone, and whether it shifts with daylight saving
- Look for piles at midnight, month-ends, and epoch defaults
- Check the sampling interval, gaps, and duplicate timestamps

IDENTIFIERS & CODES

- Never summarize them numerically: an average patient ID means nothing
- Test uniqueness, and whether IDs are reused across systems or years
- Watch for stripped leading zeros and scientific-notation damage
- Confirm that a code column matches its official code list

11 Time series: plot it, then decompose it

Structure you can see by taking the series apart

Plot every variable against time before anything else. Then check for:

- Trend, and whether it is real or a definition change
- Seasonality and cycles
- Structural breaks
- Missing or irregular periods
- Changing variance
- Level shifts after a system migration
- Day-of-week and holiday effects
- Autocorrelation and lag structure

Exploratory tools: rolling mean and rolling SD, seasonal decomposition (classical, STL, or X-13), ACF and PACF, lag plots, calendar heatmaps, year-over-year overlays.

```
from statsmodels.tsa.seasonal import STL
res = STL(s, period=52).fit()
res.plot()
s.rolling(12).mean(); s.rolling(12).std()
```

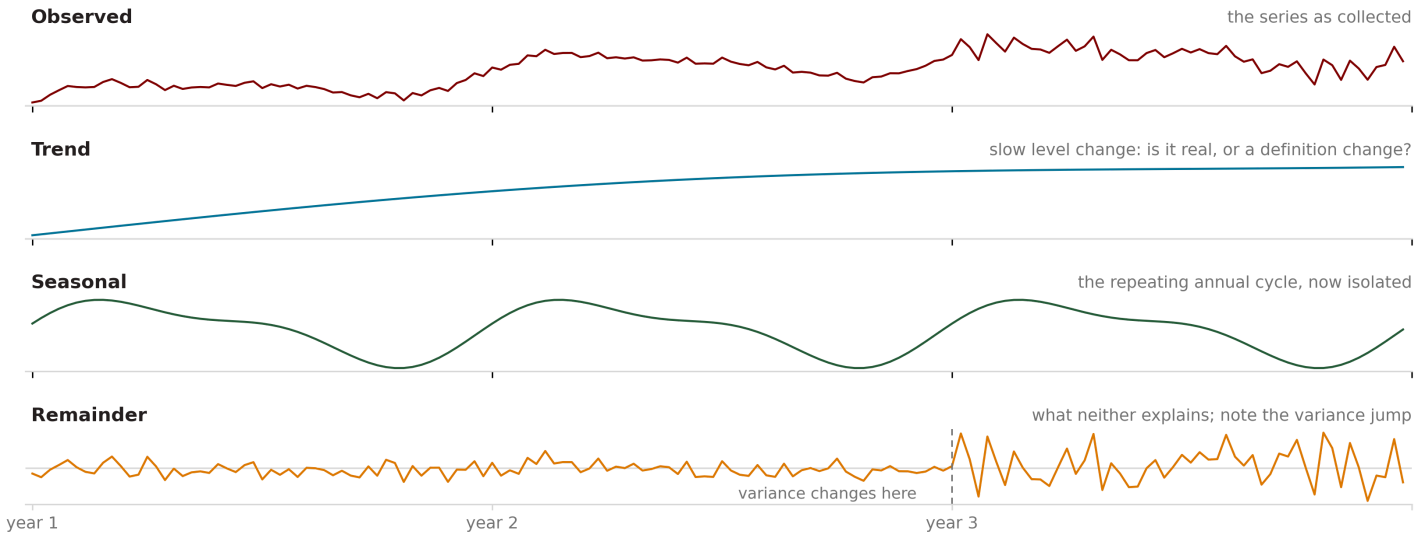
Decomposition as description

Splitting a series into **trend + seasonal + remainder** is not forecasting; it is a way of *seeing*. Each component answers a different question, and the remainder is where the surprises live.

Choose additive when the seasonal swing is roughly constant in size, multiplicative (or decompose the log) when the swing grows with the level.

Read the remainder

Spikes are events worth naming. A widening band means the process itself changed. Leftover seasonality means your period is wrong.



The same series, taken apart. The trend and the annual cycle are invisible in the raw line but obvious once separated, as is the change in variance in the final year, which no summary statistic of the whole series would reveal.

12 Space and groups

Structure that breaks the independence assumption

GEOSPATIAL

Map the raw points before mapping the rates. Evaluate geographic coverage, spatial clustering, regional gaps, spatial autocorrelation, and coordinate errors: points at (0, 0), swapped lat/long, centroids standing in for unknown addresses.

Tools: GeoPandas, Folium, Kepler.gl, PySAL, QGIS.

A choropleth of counts is usually a population map. Normalize by the right denominator, and remember that area-level patterns cannot be read as individual-level ones (the ecological fallacy).

GROUPED & HIERARCHICAL

Ask whether observations are nested:

Students → Schools → Districts · **Customers → Stores → Regions**
Patients → Hospitals · **Repeated measures → Subject**

Look within groups as well as across them: plot group-level distributions side by side, compare within-group and between-group variation, and check how much of the total spread is simply group identity.

Ignoring grouping overstates how much independent information you have and invites Simpson's paradox. A hundred measurements on ten patients is not a sample of a hundred.

13 The EDA mindset

Six questions, asked on every finding

01	02	03	04	05	06
What do I see? State the pattern plainly, without explaining it yet.	**Why might I see it?** Generate competing explanations, not one.	**Is it a data problem?** Check quality, collection, and provenance first.	**Does it make domain sense?** Ask someone who knows the subject matter.	**What should I look at next?** Design the targeted check that discriminates.	**What do I now believe?** Write it down with the evidence and the caveat.

...and the answer sends you back around. Stop when new looks stop changing what you believe.

14 Write the findings down

The part everyone skips

REPORTING BEST PRACTICES

For every finding worth keeping, record:

- **What I observed:** the pattern, in one sentence
- **The evidence:** the figure or table, by name
- **How confident:** on how many observations, and how sure you are
- **What it might mean:** competing explanations
- **What I did about it,** or why I left it alone
- **What is still open:** the questions this finding raises that you have not yet answered, and what it would take to answer them

KEEP IT REPRODUCIBLE

- Record the data version, extract date, and query
- Set and record random seeds
- Read from paths, not from a file someone edited by hand
- Turn recurring checks into functions, then assertions
- Restart and run top to bottom before you believe the notebook

BE TRANSPARENT

- Say how many observations each claim rests on
- Show what you excluded, and why
- Name the analyses that found nothing
- Distinguish what the data shows from what you suspect
- State plainly what this data cannot answer

15 Final checklist

Before you write the summary

- ☐ I can say what one observation represents.
- ☐ I know who collected this data, when, and why.
- ☐ I know which population is represented, and who is absent.
- ☐ Variable types, including ordinal, are correctly identified.
- ☐ Every variable has a definition and a plausible range.
- ☐ Missingness is quantified per variable and per subgroup.
- ☐ The missingness mechanism was argued, not assumed.
- ☐ Duplicates and impossible values were investigated.
- ☐ Every claim in my summary points to a specific figure or table.
- ☐ Every numeric variable was plotted, not just summarized.
- ☐ Summary statistics match the shape of the distribution.
- ☐ Outliers were examined and explained, not silently removed.
- ☐ Key relationships were plotted before being quantified.
- ☐ Subgroup differences and Simpson's paradox were checked.
- ☐ Temporal, spatial, and hierarchical structure was examined.
- ☐ Comparisons carry effect sizes and uncertainty, not just p-values.
- ☐ Surprises were traced to a cause or logged as open questions.
- ☐ The whole analysis can be rerun from the raw data.

THE GOLDEN RULE

Do not summarize what you have not plotted, and do not explain what you have not verified. EDA is finished when you can say how the data came to exist, where it is likely to be wrong, and which questions it can honestly answer.

16 The tools landscape

Know one deeply; know what the others are for

Tool family	Strongest at	Weakest at	Cost & learning curve
Python pandas, seaborn, Plotly	General-purpose exploration and anything custom; reproducible pipelines from raw file to finished figure	Slower for a first look than a drag-and-drop tool; plots need code before they look presentable	Free. Moderate curve; the ecosystem, not the syntax, is the hard part
R tidyverse, ggplot2	Statistical depth; the shortest path from a data frame to a well-designed plot; strong reporting via Quarto	Less common in engineering settings; memory-bound on very large data	Free. Moderate curve; grammar-of-graphics pays back quickly
Tableau	Very fast visual exploration by dragging fields; excellent maps; dashboards non-technical colleagues will actually use	Limited statistics; transformations hard to audit or version; work is trapped in the tool	Tableau Public free; Desktop licensed. Easy to start
Power BI	The same rapid visual exploration inside the Microsoft stack; Power Query cleans data with a recorded, replayable script	DAX is its own language with real depth; publishing and sharing need licenses	Desktop free on Windows; sharing licensed. Easy start, deep tail
Spreadsheets Excel, Google Sheets	Eyeballing a small file; pivot tables; quick sanity checks; universally available	Silent errors, no audit trail, sluggish long before its million-row limit, coerces types (gene names, ZIP codes, dates)	Cheap. Trivial to start, hard to keep honest
Statistical packages SPSS, Stata, SAS, JASP	Survey and panel data, weights, and complex designs handled properly; standard in many social-science and clinical settings	License cost; less flexible for custom graphics or non-tabular data	Licensed (JASP free). Menu-driven start, syntax for reproducibility
Automated profilers ydata-profiling, Sweetviz, skimpy	A complete first pass in one call: types, missingness, cardinality, distributions, correlations, duplicates	Cannot know your grain or which values are impossible; slow and unreadable on wide data	Free. Nearly no curve
Notebooks & literate tools Jupyter, Quarto, marimo, Observable	Keeping code, output, and reasoning in one narrative you can hand to someone else	Hidden state and out-of-order execution; large outputs make version control painful	Free. Small curve, real discipline
Specialist tools	OpenRefine for messy categories · missingno for missingness views · pandera for validation · QGIS for spatial · Gephi for networks	Each solves one problem well and nothing else	Mostly free. Learn on demand

Choosing, in five questions

- Where does the data live, and how big is it?
- Will this need to be rerun, and by whom?
- Who reads the output: you, an advisor, or a room?
- Does it need statistics the tool cannot express?
- What do your collaborators already have?

AI assistants

Good for drafting exploratory code, explaining unfamiliar output, and proposing checks you had not considered. They do not know how your data was generated, and they will not tell you when they are guessing. **Validate every line, statistic, and interpretation**; the reasoning has to be yours.

17 Using Claude for exploratory work

A fast collaborator, not an oracle

Where it genuinely helps

- Drafting pandas, R, or SQL for a check you can describe but not write
- Proposing data-quality checks you had not thought of
- Explaining unfamiliar fields, code sets, and statistical output
- Reviewing your code for silent assumptions

What stays yours

- **The grain.** It cannot know what one row means until you say so
- **Provenance.** It was not there when the data was collected
- **Judgment.** Which findings matter, and which are artifacts
- **The claims.** Your name goes on them, so verify them

Anatomy of a useful prompt

1 Context: what the data is, and where it came from
2 Structure: columns, dtypes, and what one row is
3 Task: one specific thing to produce
4 Constraints: library, data size, level of detail
5 Guardrails: "state your assumptions", "do not interpret yet"

HOW A SESSION SHOULD FLOW

01	02	03	04	05
You frame	**It drafts**	**You run it**	**You challenge**	**You decide**
The schema, the grain, the question. Not the whole file.	A plan or code, plus the assumptions behind it.	On your data, and read the output yourself.	Ask what would refute the reading.	Record the finding in your words, with its evidence.

18 Example prompts

Adapt the structure to your own data

Orientation

"Here are the column names, dtypes, and `describe()` output. One row is one patient-visit. List the ten checks you would run first, with the pandas code for each. Do not interpret the results yet."

States the unit of analysis and defers interpretation.

Data quality

"For each column, list the values that would be impossible given these definitions, and write assertions that would detect them."

Converts domain constraints into tests you can rerun on each extract.

Interpreting output

"Here is `describe()` for five variables. Which appear skewed, censored, or zero-inflated, and which single plot would confirm each?"

Pairs a reading with the evidence that would confirm it.

Selecting a figure

"Does wait time differ by clinic and by hour, across 40 clinics and 18 months? Propose three figures, state what each one reveals and what it obscures, then give the seaborn code."

What a figure obscures is as informative as what it displays.

Reviewing your own code

"Here is my groupby aggregation. What does it assume about missing values, duplicates, and the unit of analysis?"

Surfaces assumptions that produce plausible but incorrect results.

Alternative explanations

"I observe r = .62 between these variables in one subgroup. Give five non-causal explanations, and the check that would support or rule out each."

Apply this to any result that matches your expectations.

Domain vocabulary

"Explain what these fields represent in a hospital billing extract, and identify those that are commonly misinterpreted."

Useful for initial orientation; confirm against the data dictionary or a subject expert.

Drafting the summary

"Here are six findings and the figure supporting each. Draft a one-page summary that separates what the data shows from what I infer."

The evidence and the conclusions remain yours; the organization can be delegated.

VERIFICATION

Execute every line. Generated code can drop rows or coerce types without warning; compare row counts before and after.

Recompute independently. Reproduce at least one reported quantity yourself before it enters a written summary.

Ask what was assumed, then request the strongest argument against the conclusion.

Where code execution is available, require computed rather than described output; otherwise any reported quantity is an approximation. Per your data-use agreement, share schemas and summaries, not identifiable records.

19 Choosing where to work with Claude

The surface should match the task

Surface	What it is	Where it fits in EDA	Practical notes
Chat web, desktop, mobile	A conversation with file upload and a sandbox that runs Python or JavaScript and can return files	Single-file exploration: computed summaries, quick figures, interpretation of output, planning the next set of checks	Up to 20 uploads per chat; code execution has a lower per-file ceiling (30 MB at the time of writing). The sandbox does not persist between conversations
Projects	A workspace with its own chat history, knowledge base, and standing instructions	Holding the data dictionary, codebook, and analytic conventions so every session begins informed	Paid plans extend knowledge capacity through retrieval; Team and Enterprise plans allow projects to be shared
Cowork	An agentic system that plans and executes multi-step work in a cloud sandbox, with access to folders you grant on desktop	Repeated passes over a folder of extracts; producing spreadsheets, documents, and reports; recurring scheduled checks	Sessions persist across devices but cannot be shared, chat history does not carry over, and usage is higher than in chat
Claude Code	The same agentic execution in a terminal, working against a repository	Analysis you intend to keep: notebooks, scripts, validation tests, and version-controlled figures	Appropriate where the work must be reproducible and reviewable by others
Connectors and MCP servers	Links between Claude and the systems where your data and tools already live	Retrieving schemas, documentation, and query results from the source rather than pasting them	Availability is administered by your organization

Plan availability and stated limits change; confirm the current figures in the product documentation before relying on them.

20 Three worked examples

The same discipline, three settings

One file, in chat

Setup. Upload `survey_2024.csv` and state the grain.

"One row is one respondent. Using code rather than estimation, compute the missingness rate for every column, list the five highest, and plot the distributions of age and income. State any assumption you had to make about a column."

Returns. Computed tables and figures you can check line by line, and the assumptions written down.

A folder of extracts, in Cowork

Setup. Grant access to the folder holding 18 monthly extracts.

"For every file in this folder, produce one data-quality report: row counts by month, missingness by column, duplicate keys, and any column whose type or set of categories changes between months. Write it as an Excel workbook, one sheet per check, with all anomalies listed on a summary sheet."

Returns. A workbook a collaborator can read, from a task you can schedule to repeat.

A repository, in Claude Code

Setup. Run it in the repository that holds the analysis.

"Read `notebooks/` and `src/`. Add pandera schema checks for the three raw tables, call them from the loading functions, and write tests that fail if a check is removed. Do not change the analysis logic."

Returns. Version-controlled validation that outlives the exploration itself.

21 Working practices

Applies to every surface

PROVIDE AT THE START OF A SESSION

- Column names and dtypes, and the data dictionary or codebook
- What one row represents, and any nesting or repeated measures
- How the data was collected, by whom, and over what period
- The question you are trying to answer
- Constraints: library, version, data size, and the level of detail you want
- A small redacted or synthetic sample instead of real records

PRACTICES THAT KEEP THE WORK SOUND

- Keep one line of inquiry per conversation; start a new one when the question changes
- Ask for a plan before code, and approve the plan first
- Require assumptions to be stated, and questions to be asked rather than guessed
- Store the data dictionary in project knowledge so it does not need repeating
- Write findings to a log file as you go, in your own words
- Re-run the analysis from the raw data before you rely on any result